Level 2
Hackathon – Fundamentals of Web Development

Student Details

Student Name	**Kalaiyarasi.G**
Register Number	**30524U18017**
Department	**Bsc Computer Science**
ID	**B4018F686C8E568B3E74424DCD89451A**
College	**M.M.E.S Women's Art's and Science College**
Date of Submission	**30.10.2026**

Git-Hub link:

College-Event-Registration-Portal

1. Problem Statement

Describe the problem addressed by the application and explain why the application is needed.

In colleges, various events such as workshops, seminars, technical competitions, cultural programs, and hackathons are conducted every year. Managing registrations for these events manually can be time-consuming and difficult.

Students may face problems in getting event details, registering for events, and receiving registration confirmation. Organizers may also find it difficult to maintain student details and manage registrations efficiently.

The College Event Registration Portal is a web-based application designed to simplify the event registration process. It provides a platform where students can view event information and register for events online. It also helps organizers manage event details and registration information efficiently.

This project aims to make college event registration easier, faster, and more organized

2. Objectives

List the main objectives of the project and explain the purpose and expected outcomes of the application.

The main objectives of the College Event Registration Portal are:

Online Registration: To allow students to register for college events through a website.

Event Information: To display event names, dates, venues, and descriptions clearly.

Easy Access: To help students access event information from anywhere using a web browser.

Time Saving: To reduce the time required for manual registration.

Data Management: To organize student registration details systematically.

User-Friendly Interface: To develop a simple and easy-to-use website.

Error Reduction: To reduce common mistakes that may occur during manual registration.

Efficient Management: To help event organizers manage registration information more easily.

Improved Communication: To make event information accessible to interested students.

Digital Process: To replace paper-based registration with an online process.

3. Architecture

Describe the application structure, user navigation flow, and technologies used, including their roles in developing and executing the application. Include an application flow diagram illustrating user interactions from the initial page to the final output.

The College Event Registration Portal follows a web-based architecture.

The application consists of a user interface, application logic, and a data management component, depending on the technologies implemented in the project.

Frontend:

The frontend is the part of the website that users can see and interact with.

HTML: HTML is used to create the structure of the website, including headings, forms, buttons, and event information.

CSS: CSS is used to design the website by adding colours, spacing, borders, layouts, and suitable fonts.

JavaScript: JavaScript is used to provide interactive features, such as form validation and responses to user actions

Workflow Steps:

Step 1: Open the Application

The user opens the College Event Registration Portal.

The home page displays the available college events.

Step 2: View Events

Students can view the list of available events.

Each event displays its name, date, time, venue, and description.

Step 3: Select an Event

Students select the event they want to participate in.

They click the Register button to continue.

Step 4: Enter Student Details

Students enter their name, register number, department, email address, and phone number.

The application checks whether the required fields are filled in.

Step 5: Submit Registration

Students submit the registration form.

The application processes the entered details.

Step 6: Display Confirmation

A confirmation message is displayed after successful registration.

Students can verify that their registration has been completed.

Technology used:

- **Technology**

HTML

- **Purpose**

Creates the structure of the website.

- **Technology**

CSS

- **Purpose**

Provides styling and layout.

- **Technology**

JavaScript

- **Purpose**

Adds interactivity and form validation.

- **Technology**

Web Browser

- **Purpose**

Displays and runs the website.

Application Workflow Diagram:

START

|

v

Open the Portal

|

v

View Available

Events

|

v

Select an Event

|

v

Click Register

|

v

Enter Student Details

|

v

Validate the Details

|

v

Submit Registration

|

v

Display Confirmation

|

v

END

4. Practical Implementation

Describe how the application was developed and explain the implementation of its main features.

The College Event Registration Portal is developed using web development technologies. The implementation focuses on creating a simple interface through which students can view event details and complete the registration process.

First, the website structure is created using HTML. The home page contains the project title, navigation options, and event information.

Next, CSS is applied to improve the appearance of the website. It provides a consistent layout, readable text, suitable colours, and properly arranged forms and buttons.

JavaScript is used to handle interactive operations. For example, it can check whether required fields have been filled in before allowing the form to be submitted.

The website is then tested in a web browser to verify that the pages display correctly and that the implemented features work as expected.

The main aim of the implementation is to provide a convenient and organized online event registration experience.

4.1 Features Implemented

List the important features implemented in the application and explain the purpose of each feature.

The following are suitable features for a College Event Registration Portal. Include only the features that are actually present in your application.

1. Home Page
The home page introduces the College Event Registration Portal. It provides the project title and navigation options to help students access the website's main sections.

2. Event Details
The event details section displays information about college events, such as event names, dates, venues, and descriptions.

3. Registration Form
The registration form allows students to enter their details, such as name, register number, department, email address, and selected event.

4. Form Validation
Form validation checks whether required fields have been completed correctly. It helps prevent incomplete information from being submitted.

5. Event Selection
Students can select the event they wish to attend from the events available on the website.

6. Registration Confirmation
After a successful registration, the application can display a confirmation message to inform the student that the form has been submitted.

7. User-Friendly Design
The website uses a simple layout, clear labels, and readable text to make navigation easier for students.

8. Responsive Layout
If responsive design has been implemented, the website can adjust its layout to suit different screen sizes, including mobile phones, tablets, and computers.

4.2 Source Code

Provide the important sections of the source code and explain their functionality.

This section explains the role of HTML, CSS, and JavaScript in the project.

HTML

Explain how HTML structures the application's content and insert the relevant HTML code.

HTML

- **HTML stands for HyperText Markup Language. It is used to create the structure of the College Event Registration Portal.**
- **HTML elements are used to display headings, event information, input fields, labels, and registration buttons.**
- **For example, the following code creates a basic registration form.**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Event Registration Portal</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header class="header">
  <h2>🎓 College Event Registration Portal</h2>
  <nav>
    <a href="#home">Home</a>
    <a href="#events">Events</a>
    <a href="#myRegistrations">My Registrations</a>
    <a href="#register">Register</a>
    <a href="#admin">Admin</a>
  </nav>
</header>

<section id="home" class="hero">
  <div>
    <h1>Register for 2026 Campus Events</h1>
    <p>Browse upcoming workshops, hackathons and cultural events.</p>
    <p>Reserve your seat in just a few clicks.</p>
    <a href="#events" class="btn">Explore Events</a>
    <a href="#myRegistrations" class="btn secondary">My Registrations</a>
  </div>
  <div class="hero-icon">📅</div>
</section>
```

```
<section id="events" class="section">
  <h1>Upcoming Events — 2026</h1>
  <p>November 2026</p>

  <input type="search" id="searchEvent"
      placeholder="Search events..." class="search">

  <div class="filters">
    <button class="filter active" data-filter="all">All Events</button>
    <button class="filter" data-filter="week">This Week</button>
    <button class="filter" data-filter="campus">On Campus</button>
  </div>

  <div class="event-grid">

    <article class="event-card" data-date="2026-11-15">
      <span class="tag blue">Workshop</span>
      <h2>Web Development Workshop</h2>
      <p>📅 15 Nov 2026 | 10 AM – 4 PM</p>
      <p>📍 Innovation Lab, CSE Building</p>
      <p>Learn HTML, CSS, JavaScript and web development.</p>
      <p>Seats available: <span class="remaining">50</span></p>
      <button class="btn register-btn"
          data-event="Web Development Workshop">Register Now</button>
    </article>

    <article class="event-card" data-date="2026-11-20">
      <span class="tag orange">Hackathon</span>
      <h2>Tech Hackathon 2026</h2>
      <p>📅 20–21 Nov 2026 | 9 AM</p>
      <p>📍 Tech Hub, Engineering Block</p>
      <p>Build innovative technology solutions in teams.</p>
      <p>Seats available: <span class="remaining">30</span></p>
      <button class="btn register-btn"
          data-event="Tech Hackathon 2026">Register Now</button>
    </article>

    <article class="event-card" data-date="2026-11-25">
      <span class="tag purple">Cultural Fest</span>
      <h2>Cultural Fest 2026</h2>
      <p>📅 25 Nov 2026 | 4 PM – 9 PM</p>
      <p>📍 Central Amphitheatre, Campus Grounds</p>
      <p>Enjoy music, dance, art, food and competitions.</p>
      <p>Seats available: <span class="remaining">200</span></p>
      <button class="btn register-btn"
          data-event="Cultural Fest 2026">Register Now</button>
    </article>

  </div>
  <p id="noEvents" hidden>No events found.</p>
</section>
```

```
<section id="register" class="section">
  <div class="form-card">
    <h1>Student Registration Form</h1>
    <p>Fill in your details to register for an event.</p>

    <div id="errorBanner" class="error-banner" hidden>
      Please complete all required fields correctly.
    </div>

    <form id="registrationForm" novalidate>
      <label for="studentName">Full Name *</label>
      <input type="text" id="studentName" placeholder="Enter your full name">
      <small id="nameError" class="error"></small>

      <label for="studentId">Student ID *</label>
      <input type="text" id="studentId" placeholder="e.g. 22CS1045">
      <small id="idError" class="error"></small>

      <label for="email">Email Address *</label>
      <input type="email" id="email" placeholder="student@college.edu">
      <small id="emailError" class="error"></small>

      <label for="phone">Phone Number *</label>
      <input type="tel" id="phone" placeholder="10-digit mobile number">
      <small id="phoneError" class="error"></small>

      <label for="college">College / University *</label>
      <input type="text" id="college" placeholder="Enter college name">
      <small id="collegeError" class="error"></small>

      <label for="department">Department *</label>
      <select id="department">
        <option value="">-- Select Department --</option>
        <option>Computer Science</option>
        <option>Information Technology</option>
        <option>Electronics and Communication</option>
        <option>Mechanical Engineering</option>
        <option>Civil Engineering</option>
        <option>Commerce</option>
        <option>Arts and Science</option>
        <option>Other</option>
      </select>
      <small id="departmentError" class="error"></small>

      <label for="eventSelect">Select Event *</label>
      <select id="eventSelect">
        <option value="">-- Choose an Event --</option>
        <option>Web Development Workshop</option>
        <option>Tech Hackathon 2026</option>
        <option>Cultural Fest 2026</option>
      </select>
      <small id="eventError" class="error"></small>
```

```
            <label for="attendees">Number of Attendees *</label>
            <input type="number" id="attendees" value="1" min="1" max="10">
            <small id="attendeesError" class="error"></small>

            <button type="submit" class="btn submit-btn">Submit Registration</button>
        </form>

        <div id="successMessage" class="success" hidden>
            <h2>✅ Registration Submitted Successfully!</h2>
            <p>Your registration has been received.</p>
            <p>Registration ID: <strong id="registrationId"></strong></p>
            <p>Student: <strong id="confirmedStudent"></strong></p>
            <p>Event: <strong id="confirmedEvent"></strong></p>
            <button type="button" id="newRegistration" class="btn">
                Register Another Student
            </button>
        </div>
    </div>
</section>

<section id="myRegistrations" class="section">
    <h1>My Registrations</h1>
    <div class="table-wrap">
        <table>
            <thead>
                <tr>
                    <th>Registration ID</th>
                    <th>Student Name</th>
                    <th>Event</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody id="myRegistrationRows"></tbody>
        </table>
    </div>
</section>

<section id="admin" class="section">
    <h1>Admin Dashboard</h1>

    <div class="stats">
        <div class="stat-card">
            <h3>Total Registrations</h3>
            <p id="totalRegistrations">0</p>
        </div>
        <div class="stat-card">
            <h3>Total Events</h3>
            <p>3</p>
        </div>
        <div class="stat-card">
            <h3>Confirmed Registrations</h3>
            <p id="confirmedRegistrations">0</p>
        </div>
```

```
    </div>

    <div class="admin-actions">
      <h2>Recent Registrations</h2>
      <button id="exportCSV" class="btn">Export CSV</button>
    </div>

    <div class="table-wrap">

          <thead>
            <tr>
```

CSS

Explain how CSS styles the application and insert the relevant CSS code.

- **CSS stands for Cascading Style Sheets.**
- **It is used to improve the appearance and layout of the website.**

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

html {
    scroll-behavior: smooth;
}

body {
    font-family: Arial, sans-serif;
    background: #f2f5fa;
    color: #172033;
    line-height: 1.6;
}

[hidden] {
    display: none !important;
}

.header {
    background: white;
    padding: 18px 5%;
```

```
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 15px;
    flex-wrap: wrap;
    box-shadow: 0 3px 12px #00000015;
}

.header h2 {
    color: #12346b;
}

nav {
    display: flex;
    gap: 18px;
    flex-wrap: wrap;
}

nav a {
    text-decoration: none;
    color: #172033;
    font-weight: bold;
}

nav a:hover {
    color: #1765ce;
}

.hero {
    width: 90%;
    margin: 28px auto;
    padding: 40px;
    border-radius: 18px;
    background: linear-gradient(120deg, #b5d2ff, #4c83db);
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 20px;
}

.hero h1 {
    font-size: clamp(28px, 4vw, 40px);
    margin-bottom: 12px;
}

.hero-icon {
    font-size: 100px;
}

.btn {
    display: inline-block;
    background: #1765ce;
    color: white;
```

```
    padding: 12px 20px;
    margin: 10px 5px 0 0;
    border: none;
    border-radius: 8px;
    text-decoration: none;
    cursor: pointer;
    font-size: 15px;
    font-weight: bold;
}

.btn:hover {
    background: #104b9e;
}

.secondary {
    background: white;
    color: #17437e;
    border: 1px solid #17437e;
}

.section {
    width: 90%;
    margin: 35px auto;
}

.section > h1 {
    margin-bottom: 12px;
    color: #122a50;
}

.search {
    width: 100%;
    padding: 14px;
    margin: 15px 0;
    border: 1px solid #d0d7e2;
    border-radius: 10px;
    font-size: 16px;
}

.filters {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
    flex-wrap: wrap;
}

.filter {
    padding: 10px 22px;
    border: none;
    border-radius: 8px;
    cursor: pointer;
    background: #e0e5ed;
    font-size: 15px;
```

```
}

.filter.active {
  background: #1765ce;
  color: white;
}

.event-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 22px;
  margin-top: 20px;
}

.event-card {
  background: white;
  padding: 22px;
  border-radius: 14px;
  box-shadow: 0 5px 18px #00000012;
  display: flex;
  flex-direction: column;
  align-items: flex-start;
}

.event-card h2 {
  margin: 15px 0;
  font-size: 22px;
}

.event-card .seats {
  margin-top: auto;
  padding-top: 10px;
}

.event-card .btn {
  margin-top: 12px;
}

.tag {
  color: white;
  padding: 5px 12px;
  border-radius: 6px;
  font-size: 14px;
}

.blue {
  background: #5396ce;
}

.orange {
  background: #ee9344;
}
```

```
.purple {
   background: #7050b9;
}

.form-card {
   max-width: 850px;
   margin: auto;
   background: white;
   padding: 35px;
   border-radius: 16px;
   box-shadow: 0 8px 25px #00000012;
}

.form-card h1 {
   text-align: center;
   color: #183c75;
   margin-bottom: 10px;
}

.form-card > p {
   text-align: center;
   margin-bottom: 25px;
}

form label {
   display: block;
   margin-top: 16px;
   font-weight: bold;
}

form input,
form select {
   width: 100%;
   padding: 12px;
   margin-top: 6px;
   border: 1px solid #cbd2dc;
   border-radius: 7px;
   font-size: 16px;
   background: #fff;
}

form input:focus,
form select:focus {
   outline: 2px solid #8cbaff;
   border-color: #1765ce;
}

.error {
   display: block;
   color: #b42318;
   font-size: 13px;
}
```

```
.invalid {
    border: 2px solid #b42318 !important;
}

.error-banner {
    background: #fde8e8;
    border: 1px solid #d94a4a;
    color: #9f1d1d;
    padding: 14px;
    margin-bottom: 15px;
    border-radius: 8px;
}

.submit-btn {
    width: 100%;
    margin-top: 25px;
    padding: 15px;
    font-size: 17px;
}

.success {
    margin-top: 20px;
    background: #e0f7e7;
    color: #176536;
    border: 1px solid #70c58c;
    border-radius: 10px;
    padding: 22px;
}

.success h2 {
    margin-bottom: 10px;
}

.stats {
    display: grid;
    grid-template-columns: repeat(3, minmax(0, 1fr));
    gap: 18px;
    margin: 25px 0;
}

.stat-card {
    padding: 22px;
    background: white;
    border-radius: 12px;
    box-shadow: 0 4px 15px #00000010;
}

.stat-card p {
    font-size: 30px;
    font-weight: bold;
    color: #1765ce;
}
```

```
.admin-actions {
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  margin: 20px 0;
}

.table-wrap {
  width: 100%;
  overflow-x: auto;
  background: white;
  border-radius: 10px;
  margin-top: 15px;
}

table {
  width: 100%;
  border-collapse: collapse;
  min-width: 600px;
}

th, td {
  padding: 12px;
  border-bottom: 1px solid #e2e6ed;
  text-align: left;
}

th {
  background: #e9eef6;
}

.note {
  color: #596579;
  margin-top: 15px;
}

footer {
  margin-top: 45px;
  background: #172d51;
  color: white;
  text-align: center;
  padding: 25px;
}

@media (max-width: 900px) {
  .event-grid {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }

  .hero {
    padding: 25px;
  }
```

```
    .hero-icon {
        font-size: 65px;
    }
}

@media (max-width: 600px) {
    .header {
        padding: 15px;
    }

    .header h2 {
        font-size: 20px;
    }

    nav {
        gap: 12px;
    }

    nav a {
        font-size: 14px;
    }

    .hero {
        width: 94%;
        padding: 22px;
    }

    .hero-icon {
        display: none;
    }

    .hero h1 {
        font-size: 28px;
    }

    .section {
        width: 94%;
    }

    .event-grid,
    .stats {
        grid-template-columns: 1fr;
    }

    .event-card {
        padding: 18px;
    }

    .form-card {
        padding: 20px;
    }
```

```
    .admin-actions {
        align-items: flex-start;
        gap: 10px;
    }
}
```

JavaScript

Explain how JavaScript implements application logic and insert the relevant JavaScript code.

- **JavaScript is used to add interactive behaviour to the College Event Registration Portal.**
- **The following example displays a message when a valid registration form is submitted.**

```
const form = document.getElementById("registrationForm");
const searchInput = document.getElementById("searchEvent");

const eventCards = [...document.querySelectorAll(".event-card")];
const filterButtons = [...document.querySelectorAll(".filter")];

let registrations = JSON.parse(
    localStorage.getItem("collegeRegistrations") || "[]"
);

let currentFilter = "all";

const fields = [
    { id: "studentName", error: "nameError", label: "Full name" },
    { id: "studentId", error: "idError", label: "Student ID" },
    { id: "email", error: "emailError", label: "Email address" },
    { id: "phone", error: "phoneError", label: "Phone number" },
    { id: "college", error: "collegeError", label: "College" },
    { id: "department", error: "departmentError", label: "Department" },
    { id: "eventSelect", error: "eventError", label: "Event" },
    { id: "attendees", error: "attendeesError", label: "Attendees" }
];

function saveRegistrations() {
    localStorage.setItem(
        "collegeRegistrations",
        JSON.stringify(registrations)
    );
}

function validateForm() {
    let valid = true;

    fields.forEach(field => {
        const input = document.getElementById(field.id);
        const error = document.getElementById(field.error);
        const value = input.value.trim();

        let message = "";

        if (!value) {
```

```
            message = field.label + " is required.";
        } else if (
            field.id === "email" &&
            !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)
        ) {
            message = "Enter a valid email address.";
        } else if (
            field.id === "phone" &&
            !/^[6-9]\d{9}$/.test(value)
        ) {
            message = "Enter a valid 10-digit Indian mobile number.";
        } else if (
            field.id === "attendees" &&
            (!Number.isInteger(Number(value)) ||
             Number(value) < 1 ||
             Number(value) > 10)
        ) {
            message = "Enter a number from 1 to 10.";
        }

        error.textContent = message;
        input.classList.toggle("invalid", Boolean(message));

        if (message) valid = false;
    });

    document.getElementById("errorBanner").hidden = valid;
    return valid;
}

fields.forEach(field => {
    const input = document.getElementById(field.id);

    input.addEventListener("input", () => {
        input.classList.remove("invalid");
        document.getElementById(field.error).textContent = "";
    });

    input.addEventListener("change", () => {
        input.classList.remove("invalid");
        document.getElementById(field.error).textContent = "";
    });
});

function renderRegistrations() {
    const adminRows = document.getElementById("adminRows");
    const myRows = document.getElementById("myRegistrationRows");

    adminRows.replaceChildren();
    myRows.replaceChildren();

    registrations.forEach((reg, index) => {
        const row = document.createElement("tr");
```

```
        [
            reg.name,
            reg.studentId,
            reg.email,
            reg.phone,
            reg.event,
            reg.department,
            String(reg.attendees),
            reg.status
        ].forEach(value => {
            const cell = document.createElement("td");
            cell.textContent = value;
            row.appendChild(cell);
        });

        const actionCell = document.createElement("td");
        const removeButton = document.createElement("button");

        removeButton.textContent = "Remove";
        removeButton.className = "btn";
        removeButton.type = "button";

        removeButton.addEventListener("click", () => {
            if (confirm("Remove this registration?")) {
                registrations.splice(index, 1);
                saveRegistrations();
                renderRegistrations();
            }
        });

        actionCell.appendChild(removeButton);
        row.appendChild(actionCell);
        adminRows.appendChild(row);

        const myRow = document.createElement("tr");

        [reg.id, reg.name, reg.event, reg.status].forEach(value => {
            const cell = document.createElement("td");
            cell.textContent = value;
            myRow.appendChild(cell);
        });

        myRows.appendChild(myRow);
    });

    document.getElementById("totalRegistrations").textContent =
        registrations.length;

    document.getElementById("confirmedRegistrations").textContent =
        registrations.filter(reg => reg.status === "Confirmed").length;
}
```

```
form.addEventListener("submit", event => {
  event.preventDefault();

  if (!validateForm()) return;

  const eventName = document.getElementById("eventSelect").value;
  const attendees = Number(
    document.getElementById("attendees").value
  );

  const eventCard = eventCards.find(card =>
    card.querySelector(".register-btn").dataset.event === eventName
  );

  const seatsElement = eventCard?.querySelector(".remaining");
  const seats = Number(seatsElement?.textContent ?? 0);

  if (!eventCard || attendees > seats) {
    alert("Not enough seats are available for this event.");
    return;
  }

  const registration = {
    id: "CER-" + Date.now().toString().slice(-8),
    name: document.getElementById("studentName").value.trim(),
    studentId: document.getElementById("studentId").value.trim(),
    email: document.getElementById("email").value.trim(),
    phone: document.getElementById("phone").value.trim(),
    college: document.getElementById("college").value.trim(),
    department: document.getElementById("department").value,
    event: eventName,
    attendees: attendees,
    status: "Confirmed"
  };

  registrations.push(registration);
  saveRegistrations();

  seatsElement.textContent = seats - attendees;

  document.getElementById("registrationId").textContent =
    registration.id;

  document.getElementById("confirmedStudent").textContent =
    registration.name;

  document.getElementById("confirmedEvent").textContent =
    registration.event;

  form.hidden = true;
  document.getElementById("successMessage").hidden = false;

  renderRegistrations();
```

```
});

document.getElementById("newRegistration").addEventListener("click", () => {
  form.reset();
  form.hidden = false;
  document.getElementById("successMessage").hidden = true;
  document.getElementById("errorBanner").hidden = true;

  fields.forEach(field => {
    document.getElementById(field.error).textContent = "";
    document.getElementById(field.id).classList.remove("invalid");
  });

  document.getElementById("register").scrollIntoView({
    behavior: "smooth"
  });
});

document.querySelectorAll(".register-btn").forEach(button => {
  button.addEventListener("click", () => {
    document.getElementById("eventSelect").value =
      button.dataset.event;

    document.getElementById("register").scrollIntoView({
      behavior: "smooth"
    });
  });
});

function filterEvents() {
  const query = searchInput.value.toLowerCase();

  eventCards.forEach(card => {
    const matchesSearch = card.textContent.toLowerCase().includes(query);
    const date = new Date(card.dataset.date + "T00:00:00");
    const today = new Date();
    today.setHours(0, 0, 0, 0);

    const daysAway = Math.floor(
      (date - today) / (1000 * 60 * 60 * 24)
    );

    const matchesFilter =
      currentFilter === "all" ||
      currentFilter === "campus" ||
      (currentFilter === "week" && daysAway >= 0 && daysAway <= 7);

    card.hidden = !(matchesSearch && matchesFilter);
  });

  document.getElementById("noEvents").hidden =
    eventCards.some(card => !card.hidden);
}
```

```
searchInput.addEventListener("input", filterEvents);

filterButtons.forEach(button => {
  button.addEventListener("click", () => {
    currentFilter = button.dataset.filter;

    filterButtons.forEach(item =>
      item.classList.toggle("active", item === button)
    );

    filterEvents();
  });
});

document.getElementById("exportCSV").addEventListener("click", () => {
  const headers = [
    "Registration ID", "Name", "Student ID", "Email",
    "Phone", "College", "Department", "Event",
    "Attendees", "Status"
  ];

  const escapeCSV = value =>
    '"' + String(value).replace(/"/g, '""') + '"';

  const rows = registrations.map(reg => [
    reg.id, reg.name, reg.studentId, reg.email,
    reg.phone, reg.college, reg.department,
    reg.event, reg.attendees, reg.status
  ]);

  const csv = [headers, ...rows]
    .map(row => row.map(escapeCSV).join(","))
    .join("\r\n");

  const blob = new Blob(["\uFEFF" + csv], {
    type: "text/csv;charset=utf-8;"
  });

  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");

  link.href = url;
  link.download = "college-registrations.csv";
  link.click();

  URL.revokeObjectURL(url);
});

renderRegistrations();
filterEvents();
```

4.3 Project Execution

Explain how to run the application and describe its working process from user input to output.

The College Event Registration Portal can be executed in a web browser when its required HTML, CSS, and JavaScript files are available.

Steps to Execute the Project:

1.Create a project folder named College_Event_Registration_Portal.

2.Save the HTML code in a file named index.html.

3.Save the CSS code in a file named style.css.

4.Save the JavaScript code in a file named script.js.

5.Ensure that all three files are saved in the same folder.

6.Open the index.html file using a web browser.

7.Check whether the home page and registration form are displayed correctly.

8.Enter the required student details.

9.Select an event and submit the registration form.

10.Verify that the expected validation or confirmation message appears.

Working Process

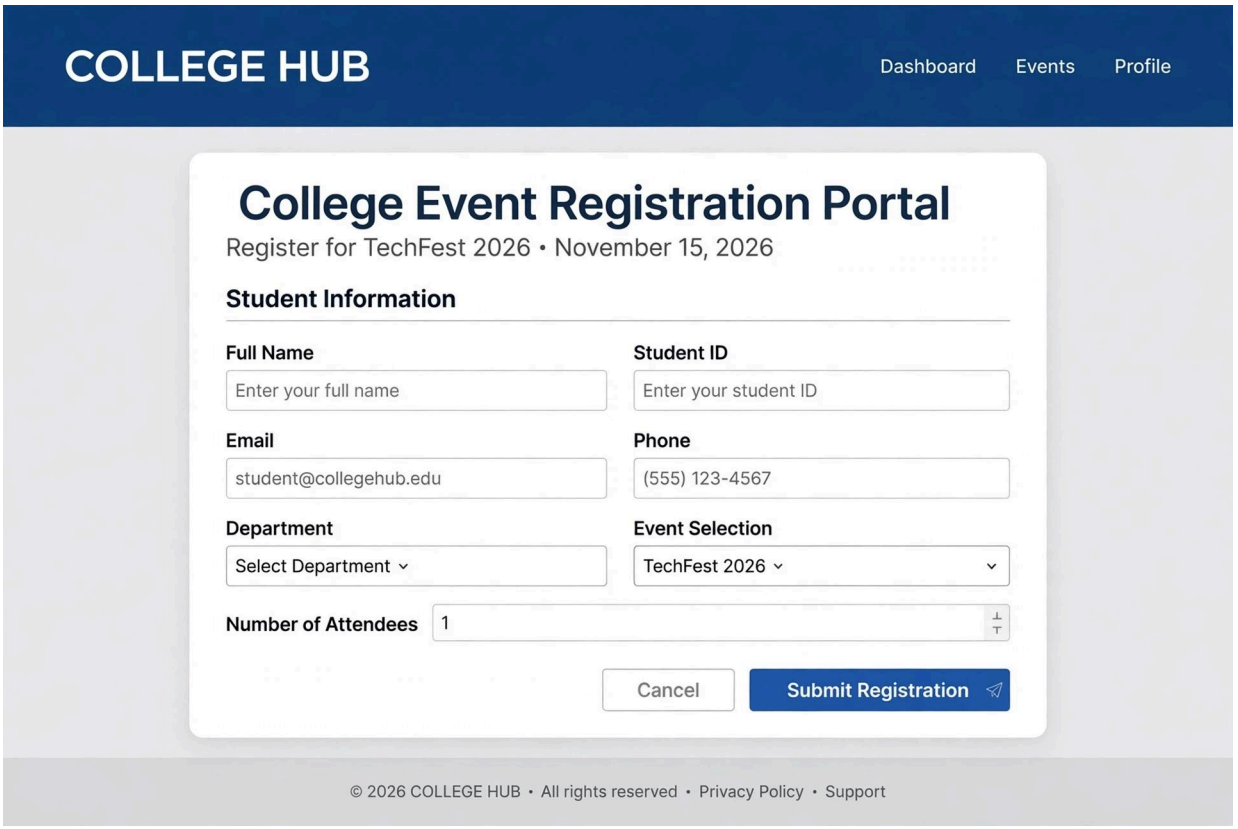
When the student opens the website, the browser loads the HTML structure and applies the CSS styling. JavaScript handles the form submission and validates the required information.

When the student submits the form, the application displays the appropriate message based on the implemented logic.

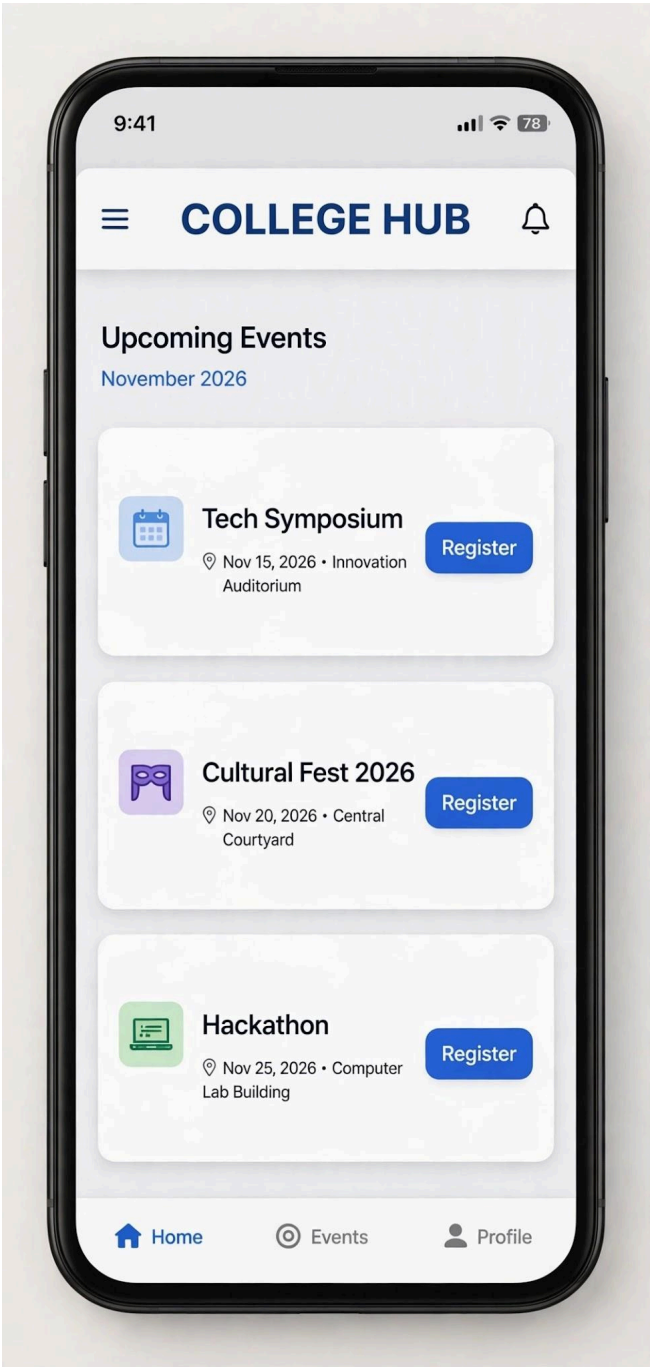
The application should be tested with both complete and incomplete form entries to check whether the validation works correctly.

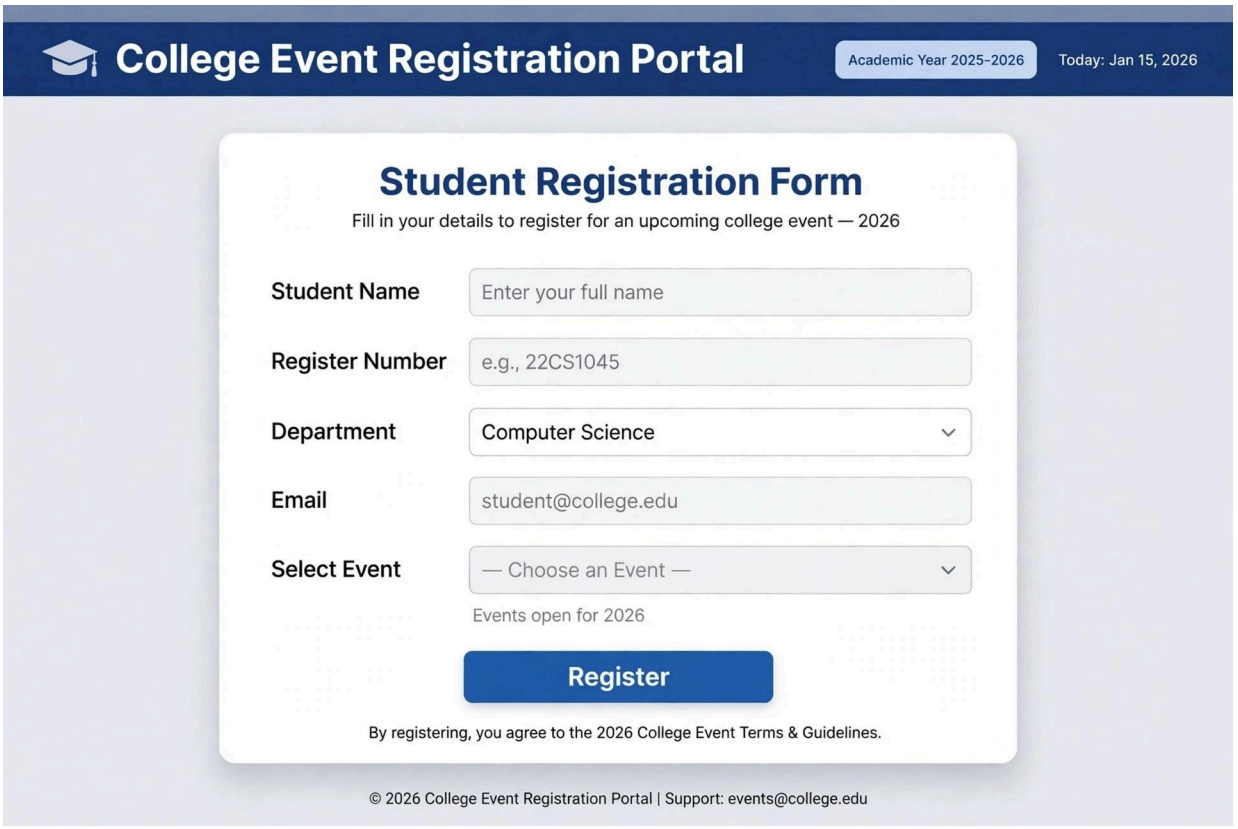
5. Output Screenshots

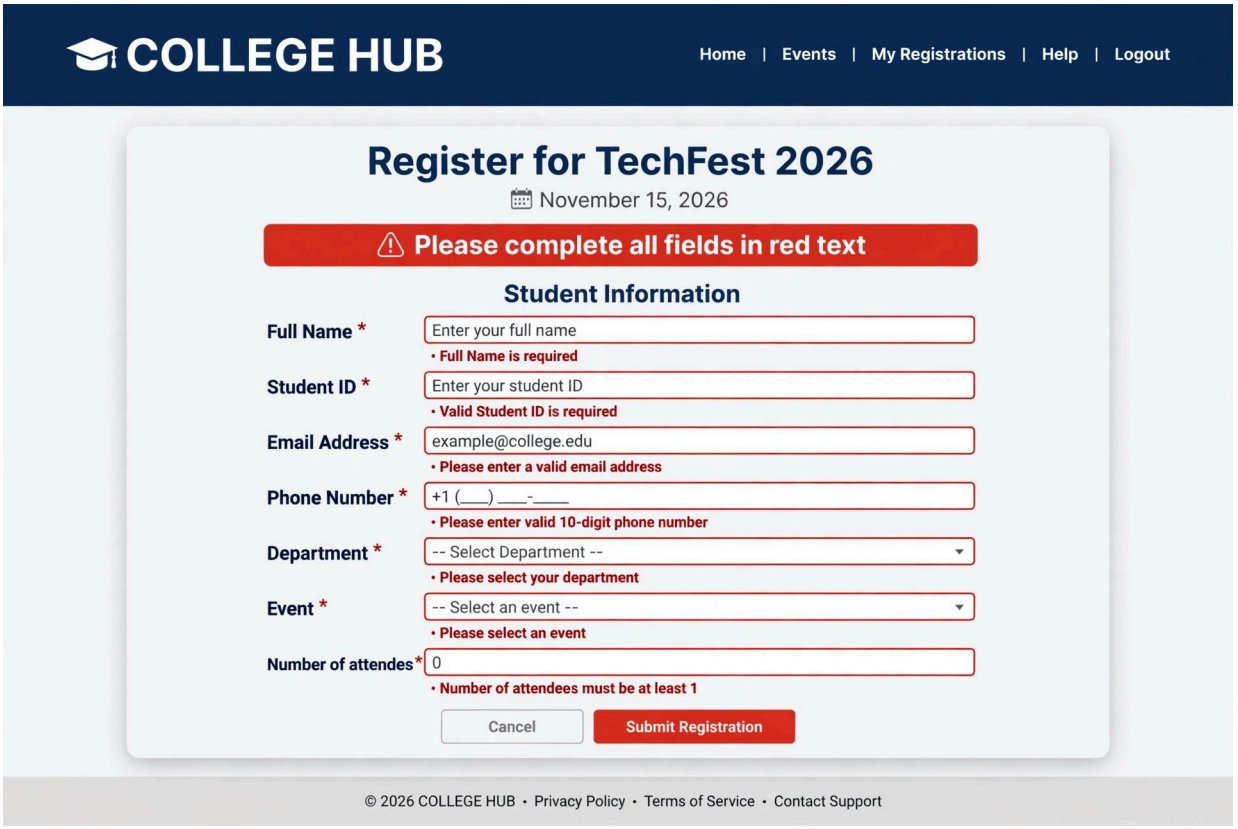
Include screenshots of the working application to demonstrate its interface, implemented features, and final results.

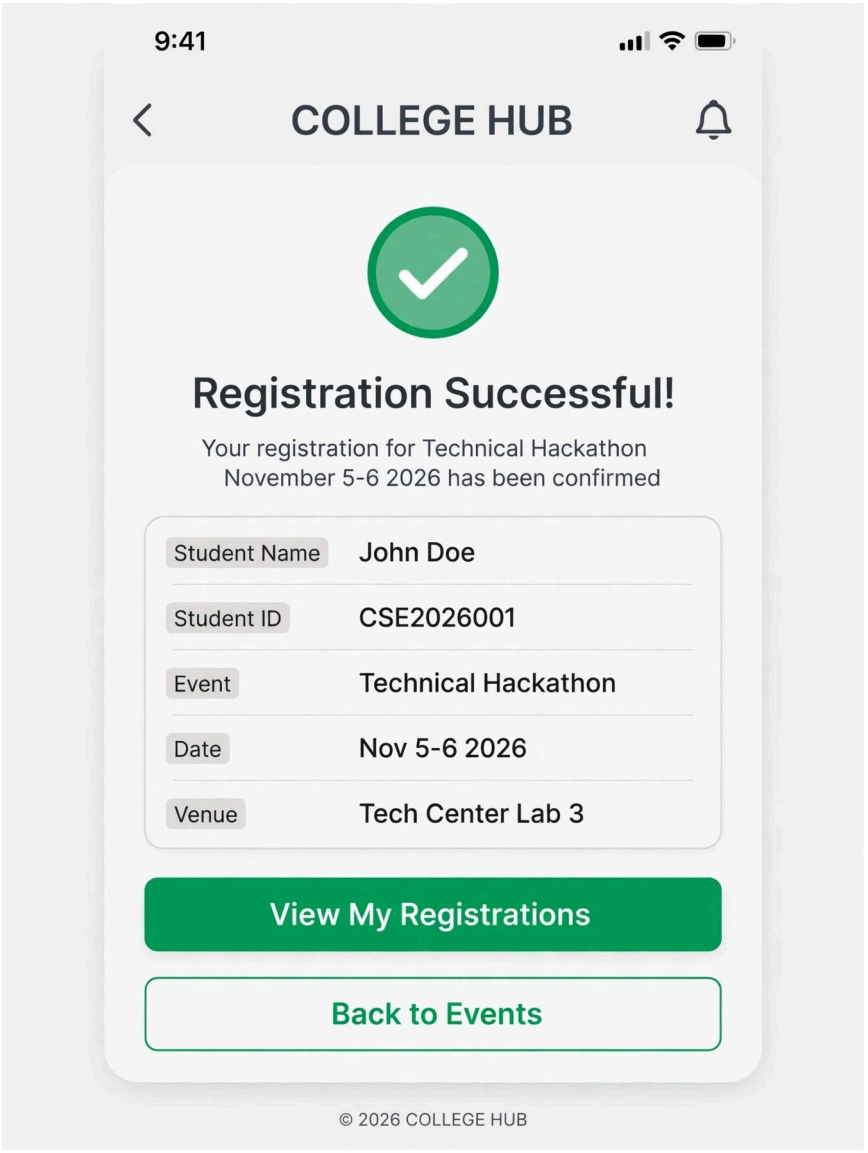


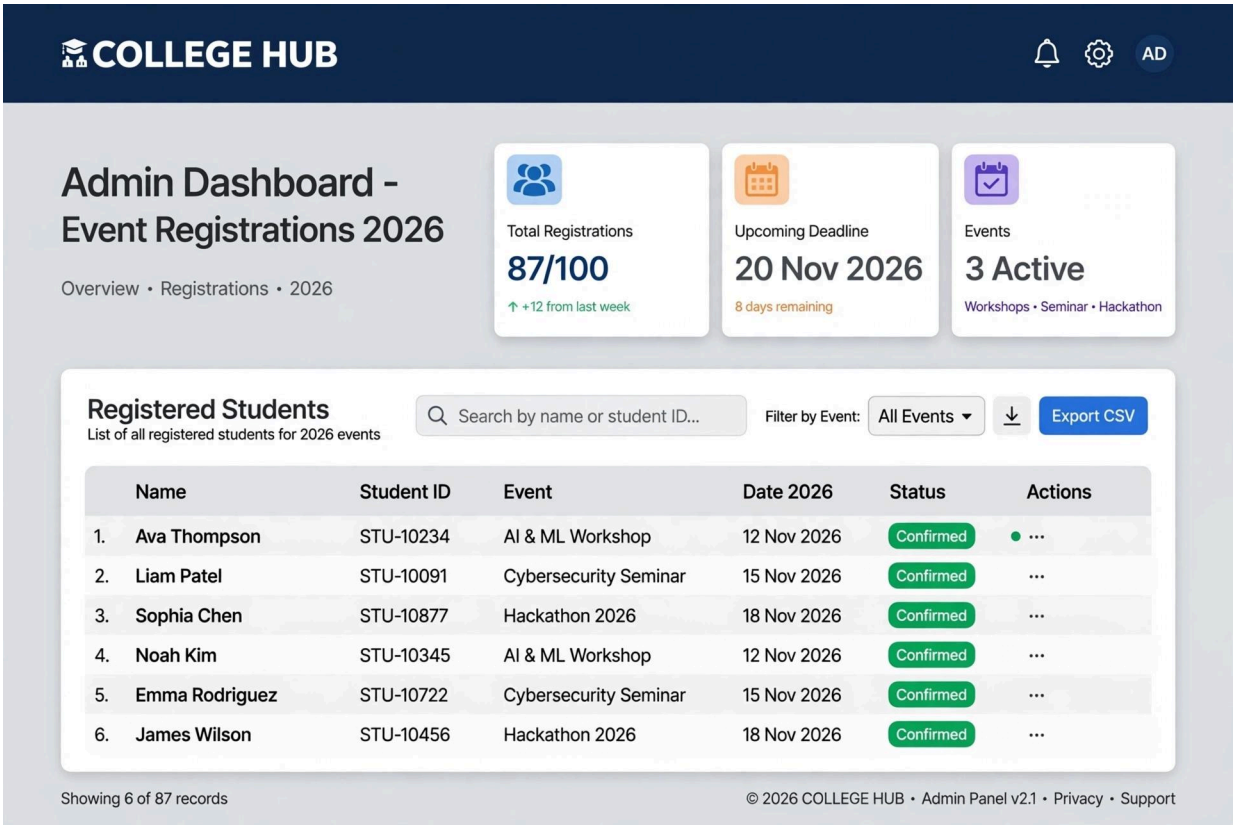
© 2026 COLLEGE HUB • All rights reserved • Privacy Policy • Support


9:41
78
COLLEGE HUB
Upcoming Events
November 2026
Tech Symposium
Nov 15, 2026 • Innovation Auditorium
Register
Cultural Fest 2026
Nov 20, 2026 • Central Courtyard
Register
Hackathon
Nov 25, 2026 • Computer Lab Building
Register
Home
Events
Profile


College Event Registration Portal
Academic Year 2025-2026
Today: Jan 15, 2026
Student Registration Form
Fill in your details to register for an upcoming college event — 2026
Student Name
Enter your full name
Register Number
e.g., 22CS1045
Department
Computer Science
Email
student@college.edu
Select Event
— Choose an Event —
Events open for 2026
Register
By registering, you agree to the 2026 College Event Terms & Guidelines.
© 2026 College Event Registration Portal | Support: events@college.edu


COLLEGE HUB
Home | Events | My Registrations | Help | Logout
Register for TechFest 2026
November 15, 2026
Please complete all fields in red text
Student Information
Full Name *
Enter your full name
• Full Name is required
Student ID *
Enter your student ID
• Valid Student ID is required
Email Address *
example@college.edu
• Please enter a valid email address
Phone Number *
+1 (___) ___-____
• Please enter valid 10-digit phone number
Department *
-- Select Department --
• Please select your department
Event *
-- Select an event --
• Please select an event
Number of attendes*
0
• Number of attendees must be at least 1
Cancel
Submit Registration
© 2026 COLLEGE HUB • Privacy Policy • Terms of Service • Contact Support


9:41
COLLEGE HUB
Registration Successful!
Your registration for Technical Hackathon
November 5-6 2026 has been confirmed
Student Name
John Doe
Student ID
CSE2026001
Event
Technical Hackathon
Date
Nov 5-6 2026
Venue
Tech Center Lab 3
View My Registrations
Back to Events
© 2026 COLLEGE HUB



6. Technical Explanation

Provide a technical overview of the technologies, programming concepts, and logic used in the application.

The College Event Registration Portal demonstrates the basic concepts of web development through the integration of HTML, CSS, and JavaScript.

1. HTML Structure

HTML provides the basic structure of the website. It organizes headings, paragraphs, forms, input fields, and buttons.

2. CSS Styling

CSS controls the visual appearance of the application. It improves readability, spacing, alignment, and the overall user interface.

3. JavaScript Functionality

JavaScript handles user interactions and form-related operations. It can validate input fields and display appropriate messages.

4. Form Validation

Form validation checks whether required information has been entered. It helps prevent incomplete submissions and improves the user experience.

5. Event Handling

JavaScript event handling allows the application to respond to user actions, such as clicking the registration button or submitting the form.

6. User Interface Design

The interface is designed to make the application easy to understand and navigate. Clear headings, labels, and buttons help users complete tasks.

7. Browser Execution

The website runs in a compatible web browser, which interprets the HTML, CSS, and JavaScript files to display and operate the application.

8. Data Management

Student registration details may be collected through the form. If the project includes a backend and database, these details can be stored and retrieved for registration management. If it is a frontend-only project, permanent storage should not be claimed.

7. Conclusion

Summarize the project outcomes, key achievements, concepts learned, and overall functionality of the application.

The College Event Registration Portal is a web development project designed to simplify the process of registering for college events.

The project demonstrates how HTML, CSS, and JavaScript can be used together to create an interactive and user-friendly website.

The application provides a structured interface for displaying event information and collecting student registration details. Form validation and confirmation messages can improve the registration experience when these features are implemented.

This project helps demonstrate fundamental web development concepts, including webpage structure, styling, user interaction, form handling, and browser execution.

Overall, the College Event Registration Portal provides a foundation for developing a more complete online event management system for colleges.

8. Future Enhancements

Suggest possible future improvements, such as adding new features, integration, enhanced security, and improved user experience.

The College Event Registration Portal can be improved in several ways in the future.

Student Login: Add a secure login system so students can access their accounts.

Admin Dashboard: Provide an administrator dashboard for creating, updating, and managing events.

Database Integration: Add a database to store and retrieve student registration details.

Email Confirmation: Send confirmation emails to students after successful registration.

Event Capacity Management: Limit registrations based on the maximum number of participants allowed for each event.

Registration Status: Allow students to check whether their registration has been completed.

Search and Filter: Add options to search for events by name, date, category, or department.

Responsive Design: Improve the website so it works effectively on mobile phones, tablets, and computers.

QR Code Check-In: Generate QR codes for registered students to support event attendance management.

Security Improvements: Add suitable server-side validation, secure authentication, and access controls if a backend is introduced.

Event Feedback: Allow participants to submit feedback after attending an event.

Registration Reports: Provide organizers with reports showing registration totals and event participation.

These improvements could make the portal more useful, secure, and efficient for students and college event organizers.